

Impact van ai op de software engineer

Onderzoeksverslag door Steven Heijn
September 2026



Student: Steven Heijn
Studie: Minor - Future-proof met AI
Datum: 02-09-2026

Samenvatting

Dit rapport onderzoekt wat de opkomst van slimme AI-hulpmiddelen betekent voor het werk van softwareontwikkelaars. Het onderzoek is gebaseerd op praktijktesten binnen bedrijven over langere tijd en een vragenlijst onder 415 professionals. Uit een meerjarig project waarbij een bedrijf de productiviteit wilde verdubbelen, blijkt dat het aantal ingediende code-aanpassingen (pull requests) per ontwikkelaar ruim verdubbelde (met een factor 2,09). Dit gebeurde niet van de ene op de andere dag, maar kostte zo'n negen maanden aan gewinning. Het totale aanbod aan code-aanpassingen binnen het bedrijf werd ruim drie keer zo groot, terwijl het aantal mensen dat deze code moest controleren maar met de helft toenam. Hierdoor kregen beoordelaars het twee keer zo druk en moesten ontwikkelaars een vijfde langer wachten tot hun code werd goedgekeurd. Om deze drukte op te vangen, lieten bedrijven steeds meer controles over aan automatische hulpmiddelen en bots: het aandeel menselijke beoordelingen zakte van 89% naar 68%. Wanneer er een slim systeem van meerdere AI-assistenten tegelijk werd ingezet om code te controleren, ging het verwerken van code-aanpassingen juist wél bijna een derde sneller.

AI-tools worden vooral gebruikt voor het bouwen van de voorkant van applicaties (25,2%), het oplossen van fouten (21,8%) and achtergrondsysteem (21,1%). Veelgebruikers geven aan dat ze door AI weliswaar meer regels code per dag aanpassen, maar dit leidt niet automatisch tot een beter resultaat (schijn-productiviteit). Zo zag ruim twee derde geen verbetering in het slagen van automatische testen, merkte ruim 84% dat het controleren van code net zoveel tijd kost, en heeft 65% last van vermoeidheid door het voortdurend moeten nakijken van wat de AI bedenkt. Risico's zoals verzonnen informatie (hallucinaties), fouten in bestaande software en auteursrechtproblemen worden beperkt door automatische spellings- en regelcheckers, het meegeven van de juiste achtergrondinformatie via slimme zoeksystemen en het uitschakelen van het automatisch overnemen van code. De rol van de ontwikkelaar verandert hierdoor: het gaat minder om het uit je hoofd kennen van programmeertaal en de precieze regels, en steeds meer om het overzien van het hele systeem, het grondig controleren van de uitkomsten, het goed aansturen van de AI en het behouden van een sterke basiskennis van programmeren.

Inhoudsopgave

Impact van ai op de software engineer.....	1
Samenvatting.....	2
Inhoudsopgave.....	3
Inleiding.....	3
Onderzoek.....	4
Verandering van het werkveld en de software development life cycle.....	4
Ai-tools en concepten voor productiviteit.....	6
Kwaliteit, risico's en knelpunten in de praktijk.....	7
Verschuiving van vaardigheden en competenties.....	8
Richtlijnen en strategieën voor de beroepspraktijk.....	9
Conclusie.....	11
Bronnenlijst.....	11

Inleiding

De introductie van generatieve en agentic ai-hulpmiddelen binnen software ontwikkeling leidt tot veranderingen in de manier waarop software wordt geschreven, gecontroleerd en beheerd. Waar vroeger onderzoeken vooral keken naar geïsoleerde programmeertaken in experimentele opstellingen, richten recente studies zich op langdurige metingen in operationele bedrijfsomgevingen.

Dit onderzoeksverslag behandelt de volgende hoofdvraag: welke impact heeft ai op de rol en werkwijze van de software engineer, en welke tools, concepten en competenties zijn vereist om als startend software engineer productiviteit en meerwaarde te leveren in het huidige en toekomstige werkveld?

Ter beantwoording van deze vraag zijn vijf deelvragen geformuleerd:

1. Op welke wijze is de software development life cycle en de dagelijkse taakverdeling van software engineers veranderd door de integratie van generatieve en agentic ai?
2. Welke specifieke ai-tools, technieken en concepten dragen in de praktijk aantoonbaar bij aan de productiviteit en code kwaliteit van een software engineer?
3. Welke risico's en knelpunten brengt ai-gedreven softwareontwikkeling met zich mee, en hoe mitigeren ontwikkelteams deze?
4. Welke vaardigheden en competenties worden door het werkveld als bepalend aangemerkt om als software engineer relevant en waardevol te blijven naast ai-systemen?

5. Welke aanbevelingen en richtlijnen kunnen worden geformuleerd voor software engineers om deze hulpmiddelen en vaardigheden toe te passen in de praktijk?

Het onderzoek baseert zich op empirische data uit longitudinale casestudies bij softwarebedrijven (He et al., 2026; Kumar et al., 2025), een multidimensionale survey onder 415 ontwikkelaars (Afroz et al., 2025) en methodologische literatuur over software engineering (Treude & Storey, 2025).

Onderzoek

Verandering van het werkveld en de software development life cycle

De integratie van ai verandert de taken in de software development life cycle van het handmatig intypen van programmacode naar het instrueren van modellen en het beoordelen van gegenereerde resultaten. Deze ontwikkeling beïnvloedt de doorlooptijd van taken, de taakverdeling en de werklust binnen teams.

In de longitudinale studie (He et al., 2026) naar een bedrijfsbreed mandaat om de productiviteit te verdubbelen, steeg het gemiddelde aantal samengevoegde pull requests per actieve software engineer van 21,2 per maand in de nulmeting (januari tot april 2025) naar 44,3 per maand in april 2026. Dit is een toename met een factor 2,09. Deze verandering voltrok zich geleidelijk. Bij ontwikkelaars die drie maanden actief waren met de tools lag de toename op een factor 1,51, na zes maanden op een factor 1,55 en na negen maanden op een factor 1,99.

Tijdens dezelfde periode steeg het totale volume aan pull requests binnen de organisatie met een factor 3,1, terwijl het aantal actieve reviewers slechts met een factor 1,5 groeide. De reviewlast per ontwikkelaar verdubbelde hierdoor. Pull requests met ai-gegenereerde code vroegen 20% meer tijd tussen de eerste menselijke beoordeling en de uiteindelijke samenvoeging. De totale cyclustijd van de eerste commit tot de samenvoeging nam met 22% toe. Het bedrijf paste het reviewproces aan door vaker geautomatiseerde beoordelingen in te zetten. Het aandeel pull requests met een menselijke review daalde van 89% naar 68%, terwijl het aandeel beoordelingen door reviewbots steeg van 19% naar 84%. Het percentage inhoudelijke reviews met geschreven commentaar daalde van 39% naar 21%, terwijl directe goedkeuringen zonder toelichting verdubbelden.

In de studie (Kumar et al., 2025) naar het Deputy Dev-platform onder 300 software engineers leidde de introductie van geautomatiseerde multi-agent reviews daarentegen tot kortere doorlooptijden. De totale cyclustijd van pull requests daalde met 31,8%, van 150,5 uur naar 99,6 uur. De gemiddelde tijd die nodig was voor de review zelf daalde van 128,8 uur naar 90,5 uur.



Studie	Onderzoeksgroep en methode	Productiviteit uitkomst	Gevolgen voor het reviewproces
He et al. (2026)	802 ontwikkelaars, 196.212 pull requests over 28 maanden	Toename van pull requests per ontwikkelaar met een factor 2,09	Reviewlast per persoon verdubbeld; menselijke reviewdekking gedaald van 89% naar 68%; geautomatiseerde reviews gestegen naar 84%
Kumar et al. (2025)	300 ontwikkelaars, longitudinaal praktijkonderzoek over 12 maanden	60,1% meer opgeleverde code; 28% toename in totale productiecode	31,8% daling in cyclustijd door parallele multi-agent beoordelingen
Afroz et al. (2025)	415 ontwikkelaars, survey over vijf SPACE-dimensies	Hogere aantallen commits en afgesloten taken bij frequente gebruikers	84,3% van de frequente gebruikers ervaart geen daling in reviewtijd; hogere cognitieve verificatielast

De opbrengst van ai-tools verschilt per ervaringsniveau en type codebase. Uit onderzoek (Kumar et al., 2025) bleek dat junior software engineers (SDE1) een stijging van 77,0% lieten zien in het aantal opgeleverde regels code ten opzichte van hun eerdere niveau. Bij medior engineers (SDE2) en senior engineers (SDE3) bedroeg deze stijging 44,6%. Het acceptatiepercentage van gegenereerde code lag bij juniors op 29,0%, bij mediors op 33,1% en bij seniors op 34,3%.

In de metingen (He et al., 2026) waren de verschillen tussen technische niveaus kleiner: individuele bijdragers stegen met 27%, seniors met 42%, staff engineers met 39% en principal engineers met 38%. De managementlaag liet een stijging zien van 86%, wat verklaard wordt doordat leidinggevendenden met behulp van ai gemakkelijker zelf code konden produceren. In

codebases die na 2022 zijn opgezet nam de productiviteit toe met 44%, terwijl in oudere codebases van voor 2022 de toename bleef steken op 12%, een verschil dat statistisch niet significant was.

Ai-tools en concepten voor productiviteit

De toepassing van ai varieert van automatische code-aanvulling tot autonome agents en contextintegratie via databases.

De registraties binnen het Deputy Dev-platform (Kumar et al., 2025) tonen de verdeling van het gebruik over verschillende programmeertaken:

Taakcategorie	Aandeel in gebruik	Omschrijving binnen het werkproces
Frontend- en UI-ontwikkeling	25,2%	Aanmaken van interfacecomponenten en layoutstructuren
Foutopsporing en bug fixing	21,8%	Analyseren van foutmeldingen en doorvoeren van correcties
Backend-codegeneratie	21,1%	Genereren van endpoints, serverlogica en databewerkingen
Databeheer	5,4%	Databasemigraties, queryopbouw en datamodellen
Codebegrip	4,8%	Uitleg en navigatie door bestaande functies
Refactoring	4,7%	Herstructurering van

Taakcategorie	Aandeel in gebruik	Omschrijving binnen het werkproces
		methoden zonder functionele verandering
Testautomatisering	4,3%	Genereren van geautomatiseerde unit- en integratietests
Overig (documentatie, beheer, beveiliging)	Minder dan 2% elk	Administratieve handelingen en toegangsbeheer

Ontwikkelaars gebruiken twee vormen van interactie: een chat-modus waarin oplossingen via tekst worden besproken en een act-modus waarin wijzigingen rechtstreeks op bestanden worden doorgevoerd. Bij Deputy Dev (Kumar et al., 2025) nam het percentage gekopieerde code uit de chat af van 21% naar 10 tot 12%, wat wijst op een toegenomen gebruik van de act-modus.

Om modellen van repository-informatie te voorzien, gebruikt Deputy Dev een Weaviate vector database die bestanden indexeert op basis van klassen en methoden. Via het Model Context Protocol (MCP) en language servers worden syntaxvalidaties en typecontroles uitgevoerd. Voor code reviews zet het systeem zes gespecialiseerde agents in die parallel werken: Summary, Security, Documentation, Code Maintainability, Error Detection, en Performance Optimization en Business Validation. Deze agents maken gebruik van analysehulpmiddelen: een File Reader voor regelbereiken en samenvattingen, een Path Searcher voor bestandszoekacties, een Grep Tool op basis van ripgrep voor patroonherkenning, en een Planner Tool voor complexe redeneerstappen. Een Comment Blending Engine voegt overlappende opmerkingen samen en verwijdert ongeldige suggesties.

Kwaliteit, risico's en knelpunten in de praktijk

De inzet van ai brengt risico's met zich mee voor de betrouwbaarheid van code en de mentale belasting van ontwikkelaars.

In de survey (Afroz et al., 2025) onder 415 ontwikkelaars werd het SPACE-framework toegepast om effecten op productiviteit in kaart te brengen:

- **Voldoening en welzijn:** 68,6% van de frequente gebruikers ervaart de werklust als beheersbaar, maar 65,2% rapporteert aanhoudende fysieke en mentale uitputting.
- **Prestaties:** 72,7% van de frequente gebruikers meldt een toename in het aantal gewijzigde regels code per dag. Daarentegen ziet 67,4% geen toename in het slagingspercentage van tests en rapporteert 58,6% geen versnelling in het aanleren van nieuwe API-methoden.
- **Activiteit:** Ontwikkelaars rapporteren meer commits (48,3% tegenover 7,9% bij niet-frequente gebruikers) en meer gesloten taken (55,9% tegenover 9,1%). Bij 84,3% van de frequente gebruikers nam de tijd besteed aan code reviews echter niet af.
- **Communicatie en samenwerking:** Ruim 90% van de respondenten geeft aan dat vergaderingen en e-mailactiviteiten ongewijzigd bleven.
- **Efficiëntie en flow:** Hoewel de tijd per individuele taak afnam, gaf 76,3% van de frequente gebruikers aan dat het aantal onderbrekingen niet daalde.

Deze uitkomsten duiden op spurieuze productiviteit: de tijdswinst bij het genereren van code wordt tenietgedaan door de tijd die nodig is voor het beoordelen, testen en corrigeren van gegenereerde code.

Aanvullende technische risico's uit de bronnen omvatten:

1. **Kwaliteitsdaling door automatische acceptatie:** Bij Deputy Dev leidde een optie voor automatische acceptatie van suggesties tot directe kwaliteitsvermindering, waarna deze functie werd uitgeschakeld.
2. **Verminderde prestaties in bestaande codebases:** Onderzoek (Dinh et al.) toont aan dat taalmodellen minder nauwkeurig functioneren wanneer de omliggende code fouten bevat.
3. **Licentieconflicten:** De LiCoEval-benchmark (Xu et al.) toonde aan dat modellen geregeld code overnemen uit auteursrechtelijk beschermde bronnen zonder de vereiste licentievermelding.
4. **Cognitieve schuld en intentieschuld:** Onderzoek (Treude & Storey, 2025) beschrijft dat wanneer code sneller wordt toegevoegd dan een team kan verwerken, het gedeelde begrip van het systeem afneemt, wat onderhoud op termijn bemoeilijkt.
5. **Systeemtragheid:** Onderzoekers (Kumar et al., 2025) constateerden dat responstijden van 2 tot 3 seconden te traag waren voor interactief gebruik. Responstijden onder de 500 milliseconden waren noodzakelijk om bruikbaar te zijn.

Verschuiving van vaardigheden en competenties

De competenties die nodig zijn voor software engineers veranderen door de introductie van generatieve modellen. Het accent verplaatst zich van syntactische routine naar ontwerp, evaluatie en contextdefinitie.



Onderzoekers (Treude & Storey, 2025) stellen dat de programmeur verandert van een schrijver in een orkestrator die oplossingen samenstelt en toetst. Dit omvat:

- **Orkestratie van componenten:** Ontwikkelaars sturen de samenwerking tussen agents, testsuites en repository's aan.
- **Configuratie van contextbestanden:** Projectkaders en architectuurregels worden expliciet vastgelegd in bestanden zoals agents.md en .copilot-instructions om agents binnen gestelde kaders te laten werken.
- **Vibe coding en intentiesturing:** Specificaties worden ingevoerd op basis van intenties en globale stijl, wat een verificatie achteraf vereist.
- **Ai-auditing:** Engineers moeten gegenereerde code controleren op logische fouten en hallucinaties.

Deze verschuiving brengt educatieve risico's met zich mee. Wanneer beginnende programmeurs leunen op ai-suggesties voordat zij basisprincipes beheersen, bestaat het risico dat zij conceptuele abstracties niet aanleren. In de kwalitatieve antwoorden bij de survey (Afroz et al., 2025) gaven senior ontwikkelaars aan bezorgd te zijn dat junior krachten hierdoor langer nodig hebben om een volwaardig ervaringsniveau te bereiken. Ook nam bij 58,6% van de frequente gebruikers de leersnelheid voor nieuwe API's niet toe, wat aantoont dat codegeneratie niet automatisch leidt tot kennisopbouw.

Richtlijnen en strategieën voor de beroepspraktijk

De onderzochte studies bieden handvatten om de risico's van ai-gebruik te beheersen. In de literatuur (Afroz et al., 2025) worden de ervaren knelpunten in verband gebracht met mitigerende maatregelen:

Knelpunt	Omschrijving van het risico	Mitigatiestrategie
Cognitieve belasting (Ch1)	Uitputting door het verifiëren, debuggen en corrigeren van suggesties	Organisatietraining in effectief promptgebruik en probleemaafbakening (St1)
Reviewlast (Ch2)	Verschuiving van werklast naar het controleren van andermans gegenereerde code	Gebruik ai voor een eerste filterronde, met behoud van menselijke controle voor goedkeuring (St6); kwaliteitspoorten instellen

Knelpunt	Omschrijving van het risico	Mitigatiestrategie
		voor grote wijzigingen (St7)
Productiedruk (Ch3)	Verhoogde verwachtingen vanuit management op basis van aannames over ai-snelheid	Teamnormen hanteren die ai positioneren als hulpmiddel en niet als productiedrijver (St2)
Uitvoerverbositeit (Ch4)	Grote hoeveelheden gegenereerde code en tests zonder functionele meerwaarde	Onzekerheidsindicatoren tonen en toelichtingsnormen hanteren bij pull requests (St3); projectcontext integreren (St4)
Kennishiaten (Ch5)	Afhankelijkheid van ai bij gebrek aan basiskennis bij junior engineers	Ai-gebruik combineren met leertrajecten in programmeerprincipes (St5)
Samenwerkingsafstemming (Ch6 en Ch7)	Beperkte ondersteuning voor onderlinge coördinatie en afstemming	Richtlijnen opstellen voor ai-gebruik in gedeelde communicatie-artefacten (St8)

Uit de evaluaties (He et al., 2026; Kumar et al., 2025) volgen aanvullende technische aanbevelingen voor ontwikkelaars:

- **Stuur niet enkel op samenvoegvolumes:** Het sturen op het aantal gemergedede pull requests leidt tot overbelasting van de reviewketen en oppervlakkige beoordelingen. Volumecijfers moeten gecombineerd worden met metingen van wachttijden en kwaliteitsindicatoren.
- **Hanteer lage responstijden:** De responstijd van analysetools dient onder de 500 milliseconden te blijven om de focus niet te verstoren.
- **Houd automatische code-acceptatie uitgeschakeld:** Wijzigingen moeten door een menselijke ontwikkelaar worden gecontroleerd om kwaliteitsdaling te voorkomen.

- **Koppel modellen aan projectcontext:** Voorzie modellen van repository-specifieke interfaces en ontwerprichtlijnen via vector-indexering of protocollen zoals MCP.
- **Maak ai-inbreng expliciet bij reviews:** Vermeld bij het indienen van codewijzigingen welke delen door ai zijn gegenereerd en welke verificatiestappen zijn doorlopen.

Conclusie

De beantwoording van de onderzoeksvragen op basis van de empirische literatuur leidt tot de volgende bevindingen:

1. De software development life cycle verschuift van handmatig programmeren naar het sturen, controleren en valideren van gegenereerde code. In een longitudinale bedrijfsstudie leidde dit tot een verdubbeling van het aantal pull requests per ontwikkelaar (factor 2,09), maar verdubbelde ook de reviewlast en stegen wachttijden met 20% tot 22%. Bij inzet van parallelle multi-agent reviewsystemen daalde de totale cyclustijd met 31,8%. Junior engineers realiseerden met 77% de grootste relatieve stijging in codeproductie, terwijl de winst in codebases van voor 2022 beperkt bleef tot 12%.
2. Tools en concepten die bijdragen aan de productiviteit betreffen semantische code-indexering via vector databases, protocolkoppelingen zoals het Model Context Protocol, interactie via de act-modus en parallelle agents voor gespecialiseerde reviewtaken. Het feitelijke gebruik richt zich vooral op frontend-ontwikkeling (25,2%), bug fixing (21,8%) en backend-ontwikkeling (21,1%).
3. De risico's omvatten spurieuze productiviteit, toegenomen cognitieve belasting door verificatie (ervaren door 65,2% van de frequente gebruikers), kwaliteitsverlies bij automatische acceptatie, licentieconflicten en het verlies van gedeeld systeembegrip. Teams mitigeren deze risico's door automatische acceptatie uit te schakelen, projectcontext te koppelen en kwaliteitspoorten in te stellen.
4. De vereiste competenties van software engineers verschuiven naar systeemarchitectuur, verificatie, instructieconfiguratie via bestanden zoals agents.md, en het behoud van conceptuele vakkennis om kwaliteitsverval en hiaten in leerprocessen te voorkomen.
5. Voor de beroepspraktijk geldt dat sturing niet uitsluitend op productievolumes moet plaatsvinden, dat responstijden onder de 500 milliseconden moeten blijven, dat ai-gebruik bij pull requests expliciet gedocumenteerd moet worden en dat hulpmiddelen gecombineerd moeten worden met gestructureerde scholing in programmeerprincipes.

Bronnenlijst

- Afroz, S., Feng, Z., Menezes, T., Kimura, K., Trinkenreich, B., Steinmacher, I., & Sarma, A. (2025, 28 oktober). *The Fast and Spurious: Developer Productivity with GenAI*. arXiv.org. <https://arxiv.org/abs/2510.24265>
- Dinh, T., Zhao, J., Tan, S., Negrinho, R., Lausen, L., Zha, S., & Karypis, G. (2023, 6 juni). *Large Language Models of Code Fail at Completing Code with Potential Bugs*. arXiv.org. <https://arxiv.org/abs/2306.03438>
- He, H., Agarwal, S., Denisov-Blanch, Y., Azaletskiy, P., Koyejo, S., & Vasilescu, B. (2026, 2 juli). *AI Writes Faster Than Humans Can Review: A Longitudinal Study of an Enterprise 2x Mandate*. arXiv.org. <https://arxiv.org/abs/2607.01904>
- Kumar, A., Khare, V., Sharma, D., Kumar, S., Saini, V., Yadav, A., Jain, S., Rana, A., Verma, P., Meena, V., & Edubilli, A. (2025, 24 september). *Intuition to Evidence: Measuring AI's True Impact on Developer Productivity*. arXiv.org. <https://arxiv.org/abs/2509.19708>
- Treude, C., & Storey, M. (2025, 12 februari). *Generative AI and Empirical Software Engineering: A Paradigm Shift*. arXiv.org. <https://arxiv.org/abs/2502.08108>
- Xu, W., Gao, K., He, H., & Zhou, M. (2024, 5 augustus). *LiCoEval: Evaluating LLMs on License Compliance in Code Generation*. arXiv.org. <https://arxiv.org/abs/2408.02487>